SUMMER TRAINING PROGRAM

SQL Final Project

Complete Handbook, Facilitation Guide & Submission Manual

An All-in-One Instructor & Student Guide for Planning, Designing, Building, Delivering, and Evaluating a Capstone Database Project

Prepared By: SQL Training & Corporate Data Engineering Faculty
Database Architecture & Instruction Division

Table of Contents

Right-click below and choose "Update Field" (or press Ctrl+A then F9) after opening the document, to generate a fully linked, page-numbered table of contents.

How to Use This Handbook

This edition merges the original SQL Final Project Handbook, its supplementary quick-reference pages, and the trainer's session-planning notes into a single, non-duplicated guide. It is organized around the actual life-cycle of the project:

- Part A — Foundations: why this project matters.
- Part B — Planning & Approval: choosing a project and getting it approved.
- Part C — Design Phase: requirements, ER modeling, normalization, schema.
- Part D — The Classroom Kickoff Session: the trainer's run-of-show for the in-class design day.
- Part E — Build Phase: SQL development, mandatory features, business questions.
- Part F — Reporting & Submission: report format, screenshots, folder structure, checklist.
- Part G — Evaluation Day: presentation format, viva, marking rubric.
- Part H — Wrap-Up: common mistakes, best practices, resume/GitHub, FAQ, final summary.

Boxed callouts are used consistently throughout:

- Trainer Tip — practical delivery advice for whoever is facilitating the session.
- Student Tip / Worked Example — guidance and illustrations for the person building the project.
- Common Mistake / Important Note — a frequent pitfall to actively avoid.
- Checkpoint — a short self-check to confirm a section's work is genuinely done before moving on.

Part A — Foundations

Section 1 — Why SQL Projects Matter

A final SQL project is not a formality — it is the single artifact that proves you can move from writing individual queries to designing and reasoning about a complete data system. Every idea below is something you should be able to explain confidently in an interview or a viva.

Why Companies Ask SQL Project Questions

Interviewers rarely ask candidates to recite syntax. Instead, they hand over a schema and ask you to reason about it — which foreign keys exist, why a table was split the way it was, how you would find the top-performing region. A project you built yourself, end to end, is the only way to answer these questions with genuine confidence rather than memorized answers.

Importance of Database Thinking

"Database thinking" means naturally asking: What entity does this represent? What uniquely identifies it? What depends on what? This habit of mind is what separates someone who can write a SELECT statement from someone who can design a system that a business can actually run on.

Real-World Business Applications

Every business process — sales, hiring, shipping, billing — is ultimately a set of related tables and the queries that answer questions about them. Your project should mirror this reality: it should model an operational process, not just a loose collection of tables built to showcase syntax.

Portfolio, Resume, and Interview Value

- Portfolio value: a well-documented project with an ER diagram, normalized schema, and real queries is far more convincing than a certificate alone.
- Resume value: a project lets you write specific, measurable bullet points instead of generic claims.
- Interview value: you can walk an interviewer through a real design decision you made, which is far stronger than describing a tutorial you followed.

> **CHECKPOINT — Section 1**
>
> You can explain, in one sentence, why a recruiter would care about a database project over a syntax quiz.
>
> You understand that this handbook will not hand you a finished project — you will design your own.

Part B — Planning & Approval

Section 2 — Project Deliverables & Minimum Requirements

Before choosing a topic, know exactly what the finished submission must contain. Every item below is mandatory unless marked optional.

Deliverable	Mandatory
Problem Statement	Yes
Objectives	Yes
Business Rules	Yes
ER Diagram	Yes
Normalization (UNF → 3NF), documented	Yes
Database Schema Documentation	Yes
CREATE TABLE / SQL Script	Yes
Sample Data	Yes
25–40 Business Queries	Yes
Minimum 3 Views	Yes
Minimum 2 Stored Procedures	Yes
Minimum 2 Triggers	Yes
Minimum 5 Window Function Queries	Yes
Screenshots	Yes
Final Report (PDF/DOCX)	Yes
Presentation (PPT)	Yes
README.md	Yes
GitHub Repository	Recommended, not compulsory

Minimum Project Requirements at a Glance

- 6–10 tables (8–14 is the sweet spot for most domains — see Section 3)
- Minimum 100 sample records across all tables combined
- 25–40 SQL business queries
- Minimum 3 Views · Minimum 2 Stored Procedures · Minimum 2 Triggers · Minimum 5 Window Function queries
- Database normalized to 3NF, with the UNF → 1NF → 2NF → 3NF steps documented
- 8–10 business reports

- Proper primary keys, foreign keys, and constraints throughout
- Professional documentation with captioned screenshots

Section 3 — Project Selection Guidelines

Choosing the right project is the single decision that most affects how much you learn and how impressive your final submission looks. Spend real time here before writing a single CREATE TABLE statement.

What a Good Project Needs

- Multiple business processes — e.g., booking, billing, and inventory rather than just one flow.
- Master tables that describe stable entities (Customer, Product, Employee).
- Transaction tables that record events over time (Order, Payment, Attendance).
- Clear relationships between master and transaction data (one-to-many, many-to-many).
- At least 6–10 meaningful reports that a real manager would actually ask for.
- Business rules that create genuine constraints (e.g., "a room cannot be booked twice for overlapping dates").
- Enough analytical depth to justify GROUP BY, window functions, and multi-table joins.

Signs a Project Is Too Small

A project is too small if it has three or fewer tables, no real transaction history, and every question can be answered with a single-table SELECT. If you cannot think of at least five reports that require a JOIN, the scope needs to grow.

Signs a Project Is Unnecessarily Complex

A project is over-scoped if you find yourself designing tables you cannot populate with realistic sample data in the time available, or modeling edge cases (multi-currency, multi-language, complex approval workflows) that are not core to demonstrating the SQL concepts you were trained on. Depth in your chosen domain is more valuable than breadth across unrelated ones.

> **Trainer Tip**
>
> Pick a domain you understand from everyday life — a hospital, a college, a store — so your energy goes into database design, not researching an unfamiliar industry.

> **CHECKPOINT — Section 3**
>
> You can list at least 8–10 tables your chosen domain would realistically need.
> You can name at least 6 reports the project would need to produce.

Section 4 — Project Ideas (30 Options)

Each entry below is a starting point, not a specification — adapt scope, entities, and reports to your own judgment during Requirement Gathering (Section 8).

Project	Industry	Difficulty	Tables	SQL Concepts to Demonstrate
E-Commerce Management System	Retail/E-Commerce	Intermediate	10–14	Joins, Window Functions, Views, Triggers
Hospital Management System	Healthcare	Advanced	12–16	Self Join, Stored Procedures, Triggers
Banking System	Banking/Finance	Advanced	10–14	Triggers, Window Functions, Views
Hotel Management System	Hospitality	Intermediate	8–12	Joins, Aggregates, Views, Date Functions
Airline Reservation System	Aviation/Travel	Advanced	10–14	Window Functions (RANK), Subqueries, Triggers
Railway Reservation System	Transportation	Advanced	10–14	Correlated Subqueries, Window Functions, SPs
Movie Ticket Booking System	Entertainment	Beginner–Intermediate	8–10	Joins, GROUP BY, Views, Aggregates
Online Food Delivery System	Food-Tech	Intermediate	10–12	Joins, Window Functions, Triggers
Library Management System	Education	Beginner	6–8	Joins, Date Functions, Triggers, Views
College Management System	Education	Intermediate	10–12	Self Join, Window Functions (RANK), Views
Student Management System	Education	Beginner	6–8	Aggregates, GROUP BY, HAVING, CASE
Employee Payroll System	Human Resources	Intermediate	8–10	Stored Procedures, Window Functions, Views
HR Management System	Human Resources	Intermediate	10–12	Joins, CASE, Correlated Subqueries, Views
Inventory Management System	Manufacturing/Retail	Intermediate	8–10	Triggers, Window Functions, Subqueries
Retail Store Management System	Retail	Intermediate	8–10	Aggregates, GROUP BY, Views, CASE
Pharmacy Management System	Healthcare/Retail	Intermediate	8–10	Date Functions, Triggers, Subqueries
Gym Management System	Fitness/Wellness	Beginner	6–8	Date Functions, CASE, Aggregates
Vehicle Rental System	Transportation	Intermediate	8–10	Joins, Date Functions, Views, Window Functions

Project	Industry	Difficulty	Tables	SQL Concepts to Demonstrate
Real Estate Management System	Real Estate	Intermediate	8–10	Self Join, Subqueries, Views
NGO Donation Management System	Non-Profit	Beginner–Intermediate	6–8	Aggregates, GROUP BY, Views
Event Management System	Events/Services	Intermediate	8–10	Joins, CASE, Window Functions
Courier Management System	Logistics	Intermediate	8–10	Date Functions, Triggers, Correlated Subqueries
Insurance Management System	Insurance	Advanced	10–12	Window Functions, Stored Procedures, Views, CASE
Online Learning Platform	EdTech	Intermediate	8–10	Joins, Window Functions, Views, Aggregates
Clinic Management System	Healthcare	Beginner–Intermediate	6–8	Joins, Date Functions, Views
Restaurant Management System	Food & Beverage	Beginner–Intermediate	6–8	GROUP BY, Aggregates, CASE, Views
Tourism Management System	Travel & Tourism	Intermediate	8–10	Joins, Window Functions, Subqueries
Warehouse Management System	Logistics/Supply Chain	Advanced	10–12	Triggers, Window Functions, Correlated Subqueries
Customer Support Ticketing System	IT Services/SaaS	Intermediate	6–8	Date Functions, CASE, Window Functions, Views
Subscription Billing System	SaaS/Media	Intermediate	6–8	Window Functions, Date Functions, Views, CASE

Creating Your Own Original Project Idea

If none of the 30 ideas above fit your interest, design your own using this process:

- Pick a real process you understand — a part-time job, a family business, a club you're part of, a hobby community.
- List the people/things involved as candidate master entities (who does what, to what).
- List the events that happen over time as candidate transaction entities (what gets recorded).
- Ask: what would the owner of this process want to know every week? Each answer becomes a report.
- Check your idea against the "What a Good Project Needs" checklist in Section 3 before committing.

> **CHECKPOINT — Section 4**
> You have shortlisted 2–3 project ideas and can justify why each has enough depth.

Section 5 — Project Approval Form

Every project requires trainer approval before development begins. Fill this in during the kickoff session and submit it for sign-off.

Field	Response
Project Title	
Domain	
Team Members	Not Applicable
Problem Statement	
Expected Number of Tables	
Expected Business Reports	
Special Features Planned	
Trainer Approval	
Approval Date	
Remarks	

Important Note

No two groups/individuals may submit the same topic without explicit trainer approval — see the FAQ in Section 32.

Part C — Design Phase

Section 6 — Project Requirements

Before any table is created, define the project's requirements in writing. This document becomes the foundation your ER diagram, schema, and report will all trace back to.

Business Problem

State, in two or three sentences, what real-world problem your database solves and who experiences that problem today. Example: "Independent clinics currently track patient visits on paper, making it difficult to review a patient's history or measure doctor workload."

Objectives

- What the database should let its users do (e.g., "track every patient visit and prescription").
- What decisions it should support (e.g., "identify which doctors are overbooked").

Scope

Define what is included and, just as importantly, what is deliberately excluded. Example: "This project covers outpatient visits and billing; it does not cover inpatient ward management."

Assumptions

List anything you are assuming to keep the project realistic without over-engineering it — e.g., "each patient has exactly one primary doctor," or "payments are made in a single currency."

Business Rules

Write the constraints that make the domain behave correctly — e.g., "An appointment cannot be booked outside clinic hours," or "A book cannot be issued if all copies are already checked out." These rules will later become constraints, triggers, or validation logic.

Expected Reports

List 8–10 reports you expect the database to be able to produce, phrased as business questions, not as SQL.

Expected Users

Describe the roles who would use this system — e.g., receptionist, doctor, billing clerk, administrator — and briefly what each role needs to see or do.

Future Scope

Note 2–3 features you would add if this were a real, ongoing product (e.g., "add SMS appointment reminders"). This shows evaluators you can think beyond the current submission.

> **CHECKPOINT — Section 6**
> You have a one-page written requirements document covering all seven items above.

Section 7 — Requirement Gathering

Requirement gathering translates your written business problem into the building blocks of a schema: entities, attributes, relationships, and keys.

Identifying Entities

An entity is any distinct thing your business needs to keep information about — typically a noun in your business problem. In a Hospital Management project: Patient, Doctor, Appointment, Ward, and Bill are all entities.

Identifying Attributes

Attributes are the facts you need to store about each entity. For Patient, this might include patient_id, full_name, date_of_birth, gender, and contact_number. Ask: "What would a receptionist need to look up about this entity?"

Identifying Relationships

Relationships describe how entities interact, and each has a cardinality: one-to-one, one-to-many, or many-to-many. Example: one Doctor can have many Appointments (one-to-many); a Patient can attend many Appointments and an Appointment involves one Patient and one Doctor.

Primary, Foreign, Candidate, and Composite Keys

- Primary Key: the column (or columns) chosen to uniquely identify each row, e.g., patient_id.
- Foreign Key: a column that references another table's primary key, e.g., Appointment.doctor_id referencing Doctor.doctor_id.
- Candidate Key: any column set that could have served as the primary key, e.g., a unique email address alongside patient_id.
- Composite Key: a primary key made of two or more columns together, common in junction tables such as Enrollment(student_id, course_id).

Documenting Business Rules

For each business rule identified in Section 6, note which table(s) and column(s) will enforce it — for example, a CHECK constraint, a UNIQUE constraint, or a trigger. This mapping becomes the first draft of your schema documentation.

Worked Example

Business problem: "Track which books are issued to which members and flag overdue returns."
Entities: Book, Member, IssueRecord.
Relationship: a Member can have many IssueRecords; a Book can appear in many IssueRecords over time (many-to-many resolved through IssueRecord).
Keys: IssueRecord uses a composite or surrogate key, with foreign keys to Book and Member.

CHECKPOINT — Section 7

You have a complete entity-attribute-relationship list for your chosen project, with keys marked.

Section 8 — ER Modeling

The Entity-Relationship (ER) diagram is the visual contract for your database. It should be drawable by hand and understandable by someone who has never seen your schema.

How to Draw an ER Diagram

- Draw a rectangle for every entity identified in Section 7.
- List key attributes inside or beside each rectangle, underlining the primary key.
- Draw a diamond or labeled line between entities that have a relationship, and name the relationship with a verb (e.g., "places," "issues").
- Mark cardinality at each end of the relationship (1, 1..*, *..*).
- Mark participation (whether an entity's involvement in the relationship is mandatory or optional).
- Resolve every many-to-many relationship into a junction/associative entity.

Common ER Notation Symbols

- Rectangle — entity
- Oval — attribute (underlined oval = primary key attribute)
- Diamond — relationship
- Double rectangle — weak entity (depends on another entity to exist, e.g., a room within a hotel)
- Crow's foot or 1 / M / N labels — cardinality notation, depending on the style you choose

An ASCII or whiteboard sketch is a perfectly acceptable planning tool before you redraw the diagram formally in a tool such as draw.io, Lucidchart, or MySQL Workbench for your final submission.

ER Diagram Checklist

- Every entity from Section 7 appears exactly once.
- Every relationship has cardinality marked at both ends.
- Every many-to-many relationship has been resolved into a junction entity.
- Primary keys are visually distinguished (underlined or bolded).
- Foreign keys are traceable to the entity they reference.
- The diagram is legible when printed on a single page (or clearly split across labeled pages).

> **CHECKPOINT — Section 8**
> Your ER diagram passes every item in the checklist above.

Section 9 — Database Design

Database design turns your ER diagram into concrete tables. This is where naming, data types, and constraint decisions are made deliberately rather than left to habit.

Table Design

Each entity becomes a table; each relationship becomes either a foreign key column (for one-to-many) or a junction table (for many-to-many). Keep one table per entity — avoid combining unrelated entities into a single wide table for convenience.

Naming Conventions

Choose one convention and apply it everywhere: table names in singular PascalCase (Customer, OrderItem) or plural snake_case (customers, order_items) are both acceptable — consistency is what matters, not the specific style.

Choosing Data Types

- Use INT or BIGINT for surrogate keys and counts.
- Use VARCHAR with a realistic length for names and short text, and TEXT only for genuinely long content.
- Use DATE or DATETIME (never plain text) for dates, so date functions work correctly.
- Use DECIMAL, not FLOAT, for currency values to avoid rounding errors.
- Use BOOLEAN or a constrained small integer/enum for true/false flags.

Constraints

Every table should declare its PRIMARY KEY explicitly. Every relationship should have a FOREIGN KEY constraint. Use NOT NULL for any column that should always have a value, UNIQUE for values like email addresses, and CHECK constraints to encode simple business rules (e.g., quantity > 0).

Indexing Strategy

Plan indexes on columns you will frequently filter or join on — typically foreign keys and columns used in WHERE clauses on large tables. Avoid indexing every column: each index speeds up reads but slows down writes and uses storage.

Scalability Considerations

Even in a training project, briefly note how your design would hold up with real data volume — e.g., whether a transaction table would need partitioning by date, or whether a reporting query would need a materialized view at scale. You are not required to implement these, only to show you have considered them.

> **CHECKPOINT — Section 9**
>
> Every table has a documented data type and constraint rationale, not just a working CREATE TABLE statement.

Section 10 — Normalization

Normalization is not a step you perform mentally and skip — it must be demonstrated and documented as part of your submission. Show your work for at least one non-trivial table, from an unnormalized starting point through to Third Normal Form.

Unnormalized Form (UNF)

Start by describing your data exactly as a business user might first hand it to you — often with repeating groups, such as a single Order row that lists multiple products in one cell. Capture this as your UNF baseline.

First Normal Form (1NF)

Eliminate repeating groups and ensure every column holds a single, atomic value. The multi-product Order row becomes one row per order item instead of a comma-separated list of products.

Second Normal Form (2NF)

Applies to tables with a composite primary key: remove partial dependencies, where a non-key column depends on only part of the composite key rather than the whole key. Move such columns into the table where they truly belong.

Third Normal Form (3NF)

Remove transitive dependencies, where a non-key column depends on another non-key column rather than directly on the primary key. Example: if a table stores customer_id, customer_city, and customer_state, and state depends on city rather than directly on the customer, city and state belong in a separate lookup structure.

Documenting Each Step

For each normalization step, present: the table before the change, the anomaly it caused (update, insertion, or deletion anomaly), and the table(s) after the change. This before/after/reason structure is what evaluators look for — not just a final, already-clean schema.

> **Common Mistake**
>
> Presenting only the final 3NF schema without showing the intermediate UNF → 1NF → 2NF → 3NF steps is one of the most frequent reasons students lose marks (see Section 29).

> **CHECKPOINT — Section 10**
>
> You can show, table by table, which anomaly each normalization step removed.

Section 11 — Database Schema Documentation

Professional schema documentation lets someone unfamiliar with your project understand it without reading your SQL. Create one documentation block per table using the template below.

Attribute	What to Fill In
Table Name	The exact name used in CREATE TABLE.
Purpose	One sentence: what real-world thing this table represents.
Columns	List every column with its data type.
Primary Key	Which column(s) uniquely identify a row.
Foreign Keys	Which column(s) reference which other table(s).
Relationships	How this table relates to others (one-to-many, many-to-many, etc.).
Business Purpose	Why this table exists in terms of the business problem, not just the schema.

Example: Documented Table

Attribute	Example Value
Table Name	Appointment
Purpose	Records a scheduled visit between a patient and a doctor.
Columns	appointment_id (INT), patient_id (INT), doctor_id (INT), appointment_date (DATE), status (VARCHAR)
Primary Key	appointment_id
Foreign Keys	patient_id → Patient.patient_id; doctor_id → Doctor.doctor_id
Relationships	Many-to-one with Patient; many-to-one with Doctor
Business Purpose	Lets the clinic track doctor workload and patient visit history.

Repeat this documentation for every table in your final report (Section 21). Consistency here is what makes your submission read like professional system documentation rather than a homework file.

> **CHECKPOINT — Section 11**
> Every table in your schema has a completed documentation block using this template.

Part D — The Classroom Kickoff Session

This part is the trainer's run-of-show for the in-class portion of the project. It reflects the actual program structure: one focused classroom session for idea approval and design, followed by independent development, then submission and evaluation.

Section 12 — Program Structure Overview

Phase	Activity	Duration
Day 1 (Classroom)	Project Introduction + Project Approval + ER Diagram + Schema	2 hours
Next 5 days	Students develop the project independently	Outside class
Submission Deadline+ Evaluation Day	Submit all project files+ Presentation + Viva	1 day

- Project Session: design and approval only — not coding.
- Development Phase: students complete the project independently before the submission deadline.
- Submission: SQL script, documentation, ER diagram, screenshots, presentation, and README.
- Final Evaluation: each team gives a 10–15 minute presentation, followed by a live SQL demonstration and an individual viva. Marks are awarded based on the quality of the project, presentation, and each student's understanding.

Section 13 — Suggested 2-Hour Kickoff Agenda

Run the classroom session to this schedule. The goal is that every student leaves with an approved, designed project — not a coded one.

Time	Activity
10 min	Explain objectives, marks, deliverables
15 min	Students choose project idea
20 min	Requirement gathering
20 min	ER Diagram
20 min	Normalization
20 min	Schema Design
10 min	Review every group's idea
5 min	Explain next milestones

> **Trainer Tip**
>
> If time is tight, protect Requirement Gathering, ER Diagram and Schema Design over polish — students can refine wording later, but a shaky ER diagram derails everything downstream.

Section 14 — What Students Should Leave the Classroom With

By the end of the kickoff session, every student or team should have all of the following — and should not be expected to finish coding during those two hours.

- Approved project topic
- Problem statement
- Objectives
- Business rules
- ER diagram
- Normalized design
- Table list

Section 15 — What Students Should Complete Afterward

Tell students clearly that the remaining work is to be completed independently over the next few days:

- CREATE DATABASE
- CREATE TABLE scripts
- Insert sample data
- 25–40 business queries
- Views
- Stored Procedures
- Triggers
- Window Function queries
- Screenshots
- Final report
- PPT
- README

Part E — Build Phase

Section 16 — SQL Development Guidelines

This section is a guide to where each SQL feature belongs in your project — not a syntax refresher. You already know how to write these statements; the goal now is applying them with intent.

Structural Statements

- CREATE DATABASE — one statement, at the very start of your script, naming the project database.
- CREATE TABLE — one per entity, written in dependency order (tables with no foreign keys first).
- ALTER TABLE — used sparingly and intentionally, e.g., to add a column you discovered was missing during testing; document why the change was needed.
- Constraints — declared inline with CREATE TABLE wherever possible, so the schema file itself documents the business rules.

Data Manipulation

- INSERT — used to load realistic sample data; aim for enough rows per table that your reports produce meaningful, non-trivial results.
- UPDATE — used to demonstrate a realistic state change, e.g., marking an order as shipped.
- DELETE — used to demonstrate safe row removal with a WHERE clause, ideally showing you considered referential integrity.

Views, Indexes, Stored Procedures, Triggers

- Views: use wherever a report is queried more than once, or where hiding a complex join behind a simple name improves readability.
- Indexes: add on foreign key columns and any column used heavily in WHERE or JOIN clauses; document the intent even if you cannot measure performance at your data volume.
- Stored Procedures: use for a multi-step operation with real logic — e.g., generating an invoice by inserting a header row, calculating a total, and updating a status flag, all in one call.
- Triggers: use only where an action must happen automatically in response to a data change — e.g., decrementing stock on an INSERT into OrderItem, or logging an audit row on an UPDATE.
- Window Functions: use wherever a report needs a rank, a running total, or a row-to-row comparison while still returning every row — see Section 18.

> **CHECKPOINT — Section 16**
>
> For every SQL feature you plan to use, you can state the specific business reason it appears in your project.

Section 17 — Mandatory SQL Features Checklist

Every applicable topic from your training must appear somewhere in your project, meeting at least the minimums set in Section 2. Use this checklist while building, and again before submission, to confirm full coverage.

#	Feature	Used?	Where It Appears in My Project
1	SELECT		
2	WHERE		
3	ORDER BY		
4	DISTINCT		
5	GROUP BY		
6	HAVING		
7	Aggregate Functions (SUM, AVG, COUNT, MIN, MAX)		
8	CASE		
9	String Functions		
10	Numeric Functions		
11	Date Functions		
12	Joins (INNER / LEFT / RIGHT)		
13	Self Join		
14	Subqueries		
15	Correlated Subqueries		
16	Views (min. 3)		
17	Indexes		
18	Stored Procedures (min. 2)		
19	Triggers (min. 2)		
20	Window Functions (min. 5 queries)		
21	Normalization (documented UNF → 3NF)		
22	ER Diagram		

#	Feature	Used?	Where It Appears in My Project
23	Database Design (keys, constraints, data types)		

Important Note

A feature marked ✓ without a genuine business reason behind it will still be flagged under "misuse" in evaluation (see Section 29). The checklist tracks coverage, not just presence.

CHECKPOINT — Section 17

Every row in the checklist is marked, with a one-line note on where that feature is used.

Section 18 — Window Functions Requirements

Window functions were a core part of your training and must be demonstrated meaningfully — not just included for the checklist. For each function below, build at least one query that uses it to answer a genuine business question.

Function	What It Does	A Practical Business Use Case
ROW_NUMBER()	Assigns a unique sequential number to each row within a partition.	Paginating a customer list or picking the single most recent order per customer.
RANK()	Assigns a rank to each row within a partition, with ties sharing a rank and a gap afterward.	Ranking products by sales where tied products should share the same position.
DENSE_RANK()	Like RANK(), but without gaps after tied ranks.	Ranking students by marks where you want consecutive rank numbers.
NTILE(n)	Divides rows within a partition into n roughly equal buckets.	Splitting customers into quartiles by total spending for a loyalty program.
LEAD()	Returns a value from a following row within the same partition.	Comparing this month's sales to next month's for a trend chart.
LAG()	Returns a value from a preceding row within the same partition.	Calculating month-over-month growth by comparing to the prior month.
FIRST_VALUE()	Returns the first value in an ordered partition.	Finding a customer's first-ever order date alongside every order row.
LAST_VALUE()	Returns the last value in an ordered partition (with correct frame bounds).	Finding the most recent status of a shipment alongside its history.
SUM() OVER()	Computes a running or partitioned total without collapsing rows.	Running total of daily revenue across a month.
AVG() OVER()	Computes a running or partitioned average without collapsing rows.	Rolling 7-day average of orders for a smoother sales trend.
COUNT() OVER()	Computes a running or partitioned count without collapsing rows.	Number of orders placed by a customer so far, shown next to each order.

Mandatory Window Function Categories

Beyond individual function syntax, make sure your 5+ window-function queries collectively cover these categories — each should solve a real business problem:

- Running Total
- Ranking
- Top-N per Group
- Previous vs Current Comparison (LAG/LEAD)
- Moving Average
- Cumulative Revenue

- Percentile / Quartile Analysis (NTILE)

Trainer Tip

A single well-designed report — for example, a monthly leaderboard — can naturally use RANK(), LAG(), and SUM() OVER() together. You do not need 11 separate throwaway queries; a few rich reports are stronger than many trivial ones.

CHECKPOINT — Section 18

Every window function above appears in at least one query tied to a real business question, and all seven categories are covered.

Section 19 — Business Questions

Write 25–40 business questions your database should be able to answer, grouped by theme as below. These are examples only — write your own set tailored to your chosen project, and do NOT write the SQL solutions in this section of your report.

Sales

- What is the total sales amount for each month of the year?
- Which product category generated the highest revenue?
- What is the average order value across all transactions?
- How does this quarter's sales compare with the previous quarter?

Customers

- Who are the top 10 customers by total spending?
- How many new customers were acquired each month?
- Which customers have not made a purchase in the last 90 days?
- What is the average number of orders per customer?

Products

- Which products are the best sellers by quantity and by revenue?
- Which products have never been ordered?
- What is the average rating for each product category?

Revenue

- What is the year-over-year revenue growth?
- What is the contribution of each region to total revenue?
- What is the running total of revenue across the year?

Monthly Reports

- What does the monthly performance dashboard look like for the last 6 months?
- Which month had the highest and lowest sales, and why might that be?

Ranking

- Rank employees by total sales within each department.
- Rank products within each category by revenue using window functions.

Growth

- What is the month-over-month growth rate in orders?
- Which customer segment is growing fastest?

Inventory

- Which products are below the reorder threshold?

- What is the average number of days a product stays in stock before selling out?

Top Customers/Employees

- Who are the top 5 performing employees this quarter?
- Which sales representative closed the most high-value deals?

Profit & Trend Analysis

- What is the profit margin trend across the last 12 months?
- Are there seasonal patterns visible in the order data?
- What is the correlation between discount percentage and order volume?

> **Important Note**
>
> Aim for a spread across categories rather than 30 variations of the same "top N" question. A strong set covers trends over time, rankings, comparisons, and operational alerts (like low stock or overdue items).

> **CHECKPOINT — Section 19**
>
> You have written 25–40 original business questions grouped by theme, with no SQL solutions included.

Part F — Reporting & Submission

Section 20 — Project Report Format

Your final report should read as one coherent document, not a stitched-together collection of files. Use the section order below, with consistent heading styles throughout.

Report Section	What Belongs Here
Cover Page	Project title, your name, course/batch, institution, date.
Certificate (optional)	Only if your program provides a training completion certificate template.
Acknowledgement (optional)	Brief thanks to instructors/mentors; keep it to a few sentences.
Table of Contents	Auto-generated, matching your final section numbering.
Introduction	One page introducing the domain and why you chose it.
Business Problem / Objectives / Scope	From Section 6 — refined after your design work.
Requirement Analysis	Entities, attributes, relationships, and keys from Section 7.
Business Rules	Final list, each mapped to the constraint/trigger that enforces it.
Entities & Attributes	A clean summary table, one row per entity.
ER Diagram	Final diagram image, legible at print size, with a short caption.
Normalization	UNF → 1NF → 2NF → 3NF walkthrough for at least one table, per Section 10.
Database Schema	Full schema documentation using the template from Section 11.
SQL Implementation	Key excerpts of DDL/DML, views, procedures, triggers, and window function queries, with explanations.
Sample Data	A short excerpt of sample rows per major table (not the full dataset).
Business Queries	Your 25–40 business questions from Section 19, each with its query and result summarized.
Output Screenshots	Per the checklist in Section 21.
Challenges	2–4 genuine difficulties you faced and how you resolved them.

Report Section	What Belongs Here
Future Enhancements	From Section 6's future scope, expanded with anything you learned along the way.
Conclusion	A short reflection on what the project demonstrates about your SQL and design ability.
References	Any external material consulted.
Appendix (optional)	Full DDL script, full query list, or anything too long for the main body.

Trainer Tip

Write the Introduction and Conclusion last, after everything else is finished — they read far more confidently once you know exactly what you built.

CHECKPOINT — Section 20

Your report follows this exact section order and every optional section is either included or intentionally omitted.

Section 21 — Output Screenshots

Screenshots are your proof of execution — they show your queries actually ran against your actual data, not just that they look correct on paper.

- Database creation — the CREATE DATABASE statement and confirmation it exists.
- Tables — the full list of tables (e.g., SHOW TABLES / catalog view) and at least one DESCRIBE output.
- INSERT execution — a successful bulk or sample INSERT with a row-count confirmation.
- SELECT queries — a representative sample from your business questions, query plus result.
- JOIN outputs — at least one multi-table join with a visibly correct result set.
- Views — the CREATE VIEW statement and a SELECT from the view.
- Stored Procedures — the CALL/EXEC statement and its output.
- Triggers — a before/after comparison showing the trigger's effect (e.g., stock count before and after an order).
- Window Functions — at least one screenshot per function type from Section 18.
- Final Reports — the polished output of your key business queries, ideally formatted as you would present them.

> **Best Practice**
>
> Label every screenshot with a caption stating what it demonstrates, and keep them in the same order as they are referenced in your report text.

> **CHECKPOINT — Section 21**
>
> Every item in the list above has a captioned screenshot in your submission folder.

Section 22 — Documentation Standards

Consistent formatting is what makes a report look like professional system documentation instead of a school assignment. Apply these standards throughout your report.

Element	Standard
Headings	Use a clear, consistent hierarchy (Heading 1 for sections, Heading 2 for subsections); never bold body text in place of a real heading style.
Tables	Every table has a header row, consistent column widths, and a caption above or below it.
Code Formatting	SQL code in a monospaced font (e.g., Consolas or Courier New), keywords capitalized, indented consistently.
Page Numbering	Page numbers in the footer from the first content page onward (cover page and TOC may be excluded).
Captions	Every diagram, table, and screenshot numbered and captioned (e.g., "Figure 3: ER Diagram").
Image Quality	Screenshots at a resolution where text is legible without zooming; crop out irrelevant desktop clutter.
Fonts	One heading font and one body font throughout — avoid mixing more than two font families.
Margins	Consistent 1-inch margins throughout the document.
Consistent Styling	Colors, bullet styles, and callout box formats used the same way in every section.

CHECKPOINT — Section 22

A reviewer flipping through your report at random would find every page follows the same visual standard.

Section 23 — Submission Folder Structure & Checklist

Recommended Submission Folder Structure

- Project_Name/
  - SQL_Scripts/ → project.sql
  - Documentation/ → Project_Report.pdf
  - ER_Diagram/ → ER_Diagram.png
  - Screenshots/ → Query1.png, Query2.png, Dashboard.png, ...
  - Presentation/ → Project_Presentation.pptx
  - README.md (repository root)

Final Submission Checklist

#	Item	Included?
1	SQL Script (.sql) — full DDL and DML, runnable from a fresh database	
2	Project Report (.pdf / .docx) — per the format in Section 20	
3	ER Diagram — exported as an image and embedded in the report	
4	Database Schema documentation — per the template in Section 11	
5	Sample Data — a representative excerpt for each table	
6	Query Outputs — results for all business questions in Section 19	
7	Output Screenshots — per the checklist in Section 21	
8	Presentation (.pptx)	
9	README.md — per Section 27	
10	GitHub Repository — optional but strongly recommended (see Section 27)	

> **CHECKPOINT — Section 23**
> Every item above is checked off before you submit.

Part G — Evaluation Day

Section 24 — Suggested Presentation Format

Each group gets 10–15 minutes total, split across a project walkthrough, a live SQL demonstration, and an individual viva.

5–7 minutes — Project Presentation

They should explain:

- Project Title
- Problem Statement
- Objectives
- ER Diagram
- Database Schema
- Normalization
- Business Rules
- Demo of Database
- Sample Reports
- Advanced SQL Features Used
- Challenges Faced
- Future Scope

5–8 minutes — Live SQL Demonstration

Ask them to execute:

- 2–3 important queries
- 1 View
- 1 Stored Procedure
- 1 Trigger demonstration
- 1 Window Function query

This ensures they can actually use what they built.

Section 25 — Individual Viva

Conduct a 5–10 minute viva for each, asking each member 2–3 questions individually. This also helps identify each student's understanding in a project.

Quick-Reference Marks Distribution

A simplified, at-a-glance version of the same 100 marks, useful for a scoreboard during live evaluation:

Component	Marks
Project Planning & Documentation	10
Database Design (ER Diagram + Normalization)	20
SQL Implementation	30
Reports & Advanced SQL Features	20
Presentation	10
Individual Viva	10
Total	100

CHECKPOINT — Section 27

You have scored your own project honestly against every row before submission.

Part H — Wrap-Up

Section 26 — Common Mistakes to Avoid

These are the most frequent issues seen across past student submissions. Review this list before you consider your project finished.

1. Choosing an overly simple project

Why it happens: Students often pick a project with only two or three tables to finish quickly.

How to avoid it: Choose a domain with at least one master-transaction relationship and multiple reporting angles, as outlined in Section 3.

2. Too many or too few tables

Why it happens: Either the scope is too small to demonstrate joins meaningfully, or it is so large it can't be completed in the timeline.

How to avoid it: Aim for 8–14 well-justified tables; every table should map to a real entity, not be added just to inflate the count.

3. Poor naming conventions

Why it happens: Mixing cases, abbreviations and inconsistent pluralization makes schemas hard to read.

How to avoid it: Adopt one convention (e.g., PascalCase tables, snake_case columns) from Section 29 and apply it everywhere.

4. Missing primary/foreign keys

Why it happens: Tables are created without explicit keys, relying only on informal 'ID-looking' columns.

How to avoid it: Every table needs an explicit PRIMARY KEY, and every relationship needs an explicit FOREIGN KEY constraint.

5. Lack of normalization

Why it happens: Repeating groups and derived columns are left in tables because it 'still works'.

How to avoid it: Walk every table through the UNF → 1NF → 2NF → 3NF process in Section 10 and document each step.

6. Redundant data

Why it happens: The same fact (e.g., customer city) is stored in multiple tables and drifts out of sync.

How to avoid it: Store each fact once in its owning table and retrieve it via JOIN wherever it's needed.

7. Incorrect joins

Why it happens: Using the wrong join type causes rows to silently disappear or duplicate.

How to avoid it: Always ask what should happen to unmatched rows before choosing INNER, LEFT, RIGHT or FULL join.

8. Cartesian products

Why it happens: Forgetting a join condition multiplies row counts unexpectedly.

How to avoid it: Always verify row counts after a join match your expectation before moving on.

9. Missing GROUP BY

Why it happens: Selecting a mix of aggregated and non-aggregated columns without grouping causes errors or wrong results.

How to avoid it: Every non-aggregated column in the SELECT list must appear in the GROUP BY clause.

10. Incorrect aggregate usage

Why it happens: Applying AVG or SUM to the wrong grain of data produces misleading totals.

How to avoid it: Confirm what one row represents in the result set before aggregating.

11. Not handling NULL values

Why it happens: NULLs are ignored in comparisons and aggregates, silently skewing results.

How to avoid it: Explicitly handle NULLs using IS NULL, COALESCE, or documented business rules.

12. Unoptimized queries

Why it happens: Queries that scan entire tables for simple lookups run slowly as data grows.

How to avoid it: Add indexes on frequently filtered or joined columns, described in Section 9.

13. Using SELECT * everywhere

Why it happens: Pulling all columns wastes resources and hides intent.

How to avoid it: Select only the columns actually needed for the report or task.

14. Missing indexes

Why it happens: No indexing strategy is documented, even though it was covered in training.

How to avoid it: Identify at least two to three columns that justify indexing and document the reasoning.

15. Unnecessary triggers

Why it happens: Triggers are added just to check a box, without a genuine business reason.

How to avoid it: Only use a trigger where an automatic, event-driven action is genuinely required, such as maintaining stock levels.

16. Stored procedures without purpose

Why it happens: Procedures wrap a single trivial SELECT with no reusable logic.

How to avoid it: Use stored procedures for multi-step operations with real business logic, such as generating an invoice.

17. Misuse of window functions

Why it happens: Window functions are added superficially without a genuine ranking, running-total, or comparison need.

How to avoid it: Tie every window function to a specific business question, as required in Section 18.

18. No documentation

Why it happens: The schema and queries are submitted without any explanation of design decisions.

How to avoid it: Follow the schema documentation templates in Section 11 and the report format in Section 20.

19. No screenshots

Why it happens: Reviewers cannot verify that queries actually run and produce the claimed output.

How to avoid it: Capture and label screenshots for every major deliverable as listed in Section 21.

20. Weak business justification

Why it happens: Reports and queries exist without a clear connection to a business decision they support.

How to avoid it: For every query, be able to state in one sentence which business decision it informs.

> **CHECKPOINT — Section**
> You have checked your project against every mistake above.

Section 27 — Best Practices

Practice	Guidance
Database naming	Use a clear, descriptive database name that reflects the project domain, e.g., ecommerce_management_db.
Table naming	Use singular or plural nouns consistently (pick one) — e.g., Customer or Customers, not a mix of both.
Column naming	Use descriptive names such as order_date rather than od, and avoid reserved SQL keywords as column names.
Query formatting	Capitalize SQL keywords, indent clauses, and break long queries across multiple lines for readability.
SQL comments	Add a one-line comment above complex queries explaining the business question they answer.
Folder organization	Separate your submission into folders: /schema, /queries, /diagrams, /screenshots, /report.
Version control	Use Git to track changes to your SQL scripts, with meaningful commit messages for each milestone.
Backup strategy	Keep a dated backup of your schema and sample data before making structural changes.
Code readability	Break complex queries into CTEs or views rather than deeply nested subqueries where possible.

Practice	Guidance
Query optimization	Check that filters use indexed columns and avoid unnecessary computed columns in WHERE clauses.
Professional documentation	Keep schema documentation, ER diagrams, and query explanations in sync as the project evolves.

CHECKPOINT — Section 27

Your project script and report both reflect every practice above.

Section 28 — Resume & GitHub Guidance

A project only adds value if you can present it well beyond the classroom.

Sample Resume Project Description

Resume Bullet Example

Hospital Management Database (SQL) — Designed and implemented a normalized 13-table relational database modeling patients, doctors, appointments, and billing; built 30+ analytical queries using window functions, views, and stored procedures to surface doctor workload and revenue trends; documented full ER model and schema.

Suggested GitHub Repository Structure

- /schema — CREATE TABLE scripts and constraints
- /data — sample INSERT scripts
- /queries — organized by category (business-questions, views, procedures, triggers, window-functions)
- /diagrams — ER diagram and schema diagram image files
- /screenshots — captioned output screenshots
- /report — the final project report
- README.md — at the repository root

Recommended README.md Sections

- Project title and one-line description
- Business problem and objectives
- ER diagram (embedded image)
- Tech stack (database engine, tools used)
- How to run the scripts (setup instructions)
- Highlight queries (a few impressive examples with brief explanations)
- Key learnings

Portfolio Tips

- Pin this repository on your GitHub profile if it's your strongest project.
- Use clear, professional commit messages that show your project evolved in stages, not as one bulk upload.
- Include the ER diagram image directly in the README so it renders without opening another file.

Interview Discussion Points

- Be ready to explain one design decision you changed your mind about, and why.
- Be ready to explain a query you optimized or a mistake you caught during testing.
- Be ready to whiteboard your ER diagram from memory.

> **CHECKPOINT — Section 28**
> Your README and resume bullet are both written and specific to your actual project.

Section 29 — Final Summary

You have now moved through the complete lifecycle of a real database project: selecting a business domain with genuine depth, gathering requirements, modeling entities and relationships, normalizing your schema, designing tables with deliberate constraints and data types, and implementing everything from basic CRUD statements to views, stored procedures, triggers, and window functions — all documented and presented to a professional standard.

This is precisely the workflow used in real engineering teams before a single line of application code is written on top of a database. The habits you built here — asking what a business rule actually protects, checking whether a report answers a real question, documenting a design decision instead of just implementing it — are what will separate you in interviews, internships, and your first data role.

Keep this project. Extend it if you learn a new SQL feature later. Reference it in interviews with specific, confident detail. It is not a training exercise you complete and set aside — it is the first entry in your professional portfolio.

> **Final Trainer Note**
> The best final projects are not the ones with the most tables or the most exotic queries — they are the ones where every design decision can be explained with a clear business reason. Aim for that standard, and the marks, resume value, and interview confidence will follow.

> **CHECKPOINT — Section 29 — Final Readiness**
> Your ER diagram, schema, and SQL script are complete and internally consistent.
> Your report follows the format in Section 20 with every table documented.
> You have rehearsed your presentation (Sections 24–25) and reviewed all 30 viva questions (Section 26).
> Your submission checklist (Section 23) is fully checked off.